

Implémentation du Jeu des Amazones

Soutenance de Projet - Master 1

Équipe PDP

Université de Bordeaux

30 janvier 2026

- 1 Introduction
- 2 Présentation du Jeu
- 3 Architecture Logicielle
- 4 Structure de Données : Bitboards
- 5 Intelligence Artificielle Hybride
- 6 Jeu en Réseau
- 7 Conclusion

Présentation du Jeu des Amazones

Historique & Règles :

- Inventé en 1988 par Walter Zamkaskas
- Combine stratégie d'échecs et contrôle territorial du Go
- Jeu à information parfaite, déterministe, somme nulle
- Objectif : Bloquer l'adversaire (dernier à jouer gagne)

Pourquoi ce projet ?

Problématique

- **Rareté** : Peu d'implémentations open-source modernes
- **Absence en ligne** : Quasi inexistant sur le web (vs échecs, Go)
- **Défi technique** : PSPACE-complexe → IA avancée requise
- **Opportunité pédagogique** : Implémenter bitboards, MCTS, réseau

Notre objectif :

- Créer une implémentation complète, performante et accessible

Nature du Jeu

- Information parfaite
- Déterministe
- Somme nulle
- 2 Joueurs

Complexité

PSPACE-complète

Objectif : Le dernier joueur à pouvoir effectuer un coup gagne. (Convention normale : bloquer l'adversaire)

Spécifications Techniques :

Élément	Choix
Langage	Python 3.10+
Build	pyproject.toml
GUI	PyGObject (GTK4)
Tests	pytest ($\geq 85\%$)
Doc	Sphinx
i18n	gettext (EN/FR)
ML	Keras

Algorithme de Déplacement

Phase A : Le Mouvement

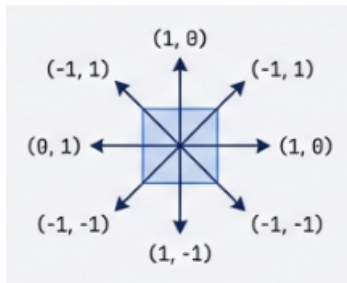
Déplacement dans une direction (Ligne, Colonne, Diagonale) vers une case **VIDE**.

Phase B : Le Tir

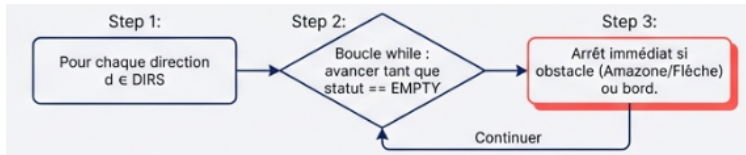
Depuis la nouvelle case, tir d'une "**flèche**" qui devient un obstacle permanent.

Logique Vectorielle (Implémentation Bitboard) :

$$DIRS = \{(1, 0), (-1, 0), (0, 1), (0, -1), (1, 1), (-1, -1), \dots\}$$



Visualisation de l'Algorithme



```
# 2. Iteration sur les 8 directions cardinales et intercardinales
for dr, dc in self.DIRECTIONS:
    # Initialisation du rayon à la première case adjacente
    curr_r, curr_c = start_row + dr, start_col + dc
    # 2. Avance itérativement tant que la case est SANS les obstacles
    while 0 <= curr_r < self.size and 0 <= curr_c < self.size:
        # Vérification de présence de la case SANS
        if self.grid[curr_r][curr_c] == 0: # 0 = vide
            # Case valide, on l'ajoute
            reachable_nodes.append((curr_r, curr_c))
        # On continue de rayon dans la même direction
        curr_r += dr
        curr_c += dc
    else:
        # 3. Condition d'arrêt : Distance rencontrée
        # la boucle s'interrompt pour cette direction (rayon)
        # la case occupée n'est PAS ajoutée.
        break
return reachable_nodes
```

Analyse des mouvements depuis (4, 4)



Nombre de coups possibles : 26
Q = Reine Active, * = Déplacement possible,
X = Flèche, N = Ennemi

Problème

Densité $> 30\%$ sur petits plateaux $\rightarrow$ Jeu bloqué.

Taille	Nb P.	Stratégie
4×4	2	Coins opposés
5×5	3	Triangle
$6 \times 6 - 11 \times 11$	4	Formule Tiers

Formule des Tiers ($N \geq 6$) :

$$T = \lfloor N/3 \rfloor$$

- **Ailes** : Ligne T , Colonnes 0 et $N - 1$
- **Centre** : Ligne 0, Colonnes T et $N - 1 - T$

Fin de Partie (F10)

- **Perdant** : Celui qui ne peut plus bouger.
- **Pas de Nul** : Jeu déterministe.
- **Plateau Scindé** : On continue (Course de remplissage).

Notation Algébrique (F20)

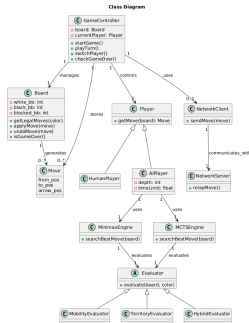
Colonnes **a-j**, Lignes **1-10**.

Exemple : e2-e4/e6

Déplacer **e2** vers **e4**, puis tirer sur **e6**.

Séparation stricte Modèle - Vue - Contrôleur

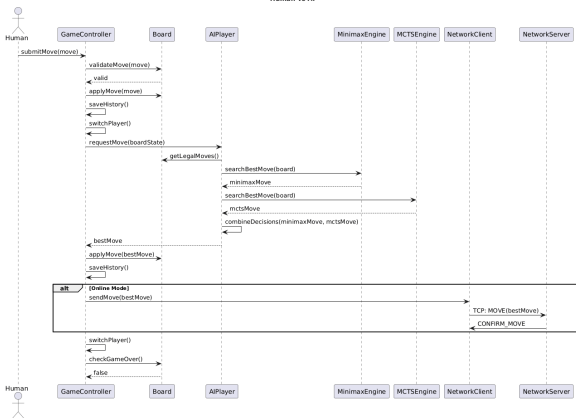
- **Modèle** : Logique pure, Bitboards, Règles. Indépendant de l'interface.
- **Vue** : Rendu graphique (PyGObject/GTK 4.x).
- **Contrôleur** : Orchestration (Réseau, Input utilisateur, IA).



Diagrammes de Séquence

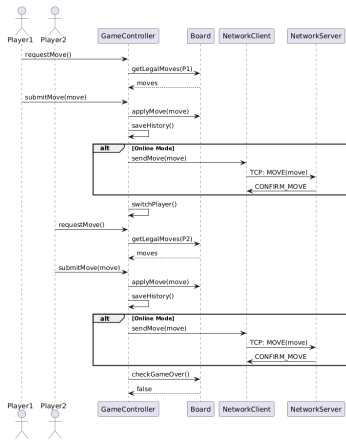
Humain vs IA

Human vs AI



Humain vs Humain

Human vs Human



Note Technique

Les exemples présentés utilisent un plateau **10×10** (configuration standard). L'implémentation finale supportera toutes les tailles de **4×4 à 11×11** en paramétrant n dans les calculs.

Optimisation par Bitboards

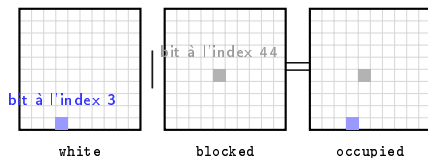
Problème : Un tableau 2D `int[10][10]` :

- Nécessite 400 octets ($100 \text{ cases} \times 4 \text{ octets/entier}$)
- Parcours $O(N^2)$ pour chercher/copier

Solution : 3 bitboards de 100 bits chacun (38 octets total !)

- `white_amazons` : 1 bitboard pour reines blanches
- `black_amazons` : 1 bitboard pour reines noires
- `blocked` : 1 bitboard pour cases bloquées
- **Gain** : $400 \rightarrow 38$ octets (**90% de réduction**)

Numérotation : Case (*col*, *ligne*) $\rightarrow$ *index* = *ligne* $\times$ 10 + *col*

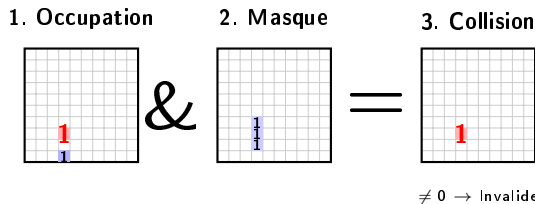


Algorithme 1 : Validation d'un Coup (Lecture)

Objectif : Vérifier si un coup est valide (ex : déplacement $d1 \rightarrow d4$)

Algorithme :

- 1 Agréger tous les obstacles via OR : occupé = white | black | blocked
- 2 Appliquer un masque du chemin voulu
- 3 Vérifier l'intersection via AND
- 4 Une valeur non nulle indique une collision



Exemple : Joueur joue $d1(3) \rightarrow d4(33)$ avec obstacle en $d2(13)$

Vérification : $(occupied \wedge (1 \ll 13)) \neq 0 \rightarrow \text{REJET}$

Message : "Coup invalide : Trajectoire bloquée en $d2$ "

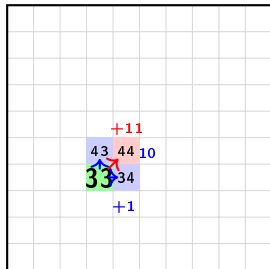
Arithmétique des Index : Calcul des Vecteurs

Principe : Sur un plateau 10×10 , index $i = \text{ligne} \times 10 + \text{colonne}$

Comment trouver les vecteurs ?

- **Verticaux :** N(+10), S(-10)
- **Horizontaux :** E(+1), O(-1)
- **Diagonaux :**

- NE(+11) = N + E
- NO(+9) = N + O
- SE(-9) = S + E
- SO(-11) = S + O



Exemple visuel :

Vérif : $33 + 10 = 43$, $33 + 1 = 34$, $33 + 11 = 44$ ✓

Algorithme 2 : Propagation (Anti-Wrapping)

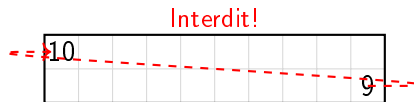
Problème du "Wrapping" : Index $9 + 1 = \text{Index } 10$ (téléportation)

Solution : Masques de bords

- MASK_COL_A : index 0, 10, ..., 90
- MASK_COL_J : index 9, 19, ..., 99

Algorithme :

- 1 $cur = P$, pour chaque direction D
- 2 Boucle tant que valide :
 - Si Est et $cur \in \text{COL_J}$ **STOP**
 - Si Ouest et $cur \in \text{COL_A}$ **STOP**
 - $next = cur + D$
 - Si $next < 0$ ou ≥ 100 **STOP**
 - Si occupé **STOP**
 - Ajouter aux coups, $cur = next$



Gestion des Erreurs Utilisateur

Le moteur rejette immédiatement tout coup invalide et affiche un message explicite.

Cas d'erreurs détectés :

① Trajectoire bloquée :

- **Erreur** : Joueur joue $d1 \rightarrow d4$ avec obstacle en $d2$
- **Détection** : $(occupied \wedge (1 \ll 13)) \neq 0$
- **Message** : *"Coup invalide : Trajectoire bloquée en d2"*

② Pas d'amazone en position de départ :

- **Erreur** : Joueur joue $e5 \rightarrow e8$ mais pas d'amazone en $e5$
- **Détection** : $(my_amazons \wedge (1 \ll 44)) = 0$
- **Message** : *"Aucune amazone en e5"*

③ Mouvement hors alignement :

- **Erreur** : Joueur joue $d1 \rightarrow f3$ (mouvement "cavalier")
- **Détection** : Pas sur même ligne/colonne/diagonale
- **Message** : *"Mouvement invalide : d1 vers f3"*

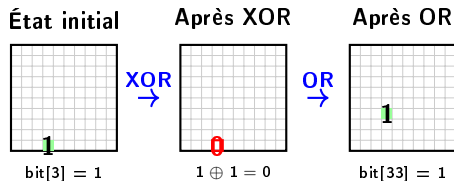
Action : Le joueur est invité à **recommencer sa saisie** sans pénalité.

Algorithme 3 : Application d'un Coup (Écriture)

Objectif : Déplacer une amazone de d1 (index 3) vers d4 (index 33)

Ordre strict : XOR $\rightarrow$ OR $\rightarrow$ OR

- 1 Libérer départ (XOR) : $\text{amazons} \oplus (1 \ll \text{from})$
- 2 Occuper arrivée (OR) : $\text{amazons} \mid (1 \ll \text{to})$
- 3 Tirer flèche (OR) : $\text{blocked} \mid (1 \ll \text{arrow})$



Algorithme 4 : Calculer les Scores

Entrée : État du plateau

Sortie : Score de chaque joueur

Algorithme (mobilité simple) :

- 1 Calculer le nombre de coups possibles pour le joueur : M_{joueur}
- 2 Calculer le nombre de coups possibles pour l'adversaire : $M_{\text{adversaire}}$
- 3 $\text{Score} = M_{\text{joueur}} - M_{\text{adversaire}}$

Interprétation :

- $\text{Score} > 0$: Avantage joueur
- $\text{Score} < 0$: Désavantage joueur
- Plus le score est élevé, plus le joueur a de mobilité

Algorithme 5 : Déterminer la Fin de Partie

Entrée : Joueur courant

Sortie : Vrai si la partie est terminée

Algorithme :

- ➊ Calculer tous les coups possibles pour le joueur courant (Opération 2)
- ➋ Si le nombre de coups est **zéro**, le joueur courant a perdu

Important :

- Le jeu des Amazones est **déterministe** et **fini**
- Le nombre de cases libres diminue strictement de 1 à chaque tour
- Le match nul est **impossible** : il existe toujours un vainqueur

Architecture hybride : Minimax + MCTS

Board $\rightarrow$ *getLegalMoves* $\rightarrow$ (*Minimax* + *MCTS*) $\rightarrow$ *Evaluator* $\rightarrow$ *bestMove*

Composants :

- **Génération** : `Board.getLegalMoves(color)`
- **Minimax** : Recherche en profondeur avec élagage $\alpha\beta$
- **MCTS** : Simulations Monte Carlo (UCT)
- **Evaluator** : 3 heuristiques (Mobilité, Territoire, Hybride)
- **Fusion** : Combinaison des propositions

Principe : Arbre alternant MAX (nous) et MIN (adversaire)

Élagage : Coupe les branches inutiles quand $\alpha \geq \beta$

Fonctions d'évaluation :

- **Mobilité** : $score = |Moves_{nous}| - |Moves_{lui}|$
- **Territoire** : $score = \#cases_{nous} - \#cases_{lui}$
- **Hybride** : $score = \lambda \cdot mob + (1 - \lambda) \cdot terr$

Monte Carlo Tree Search (MCTS)

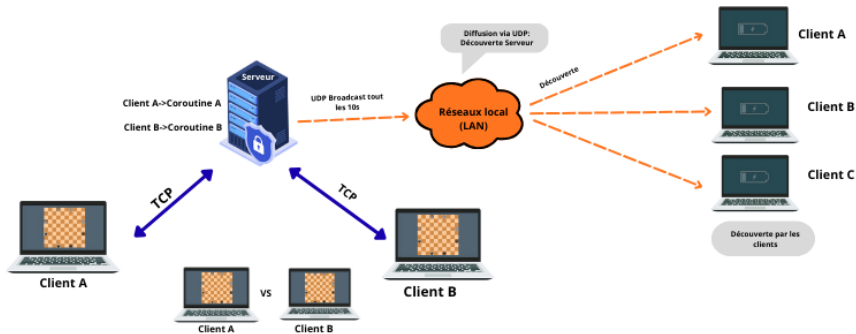
4 Phases :

- 1 **Sélection** : $UCT = \frac{w_i}{n_i} + C \sqrt{\frac{\ln N}{n_i}}$
- 2 **Expansion** : Ajouter un nœud fils
- 3 **Simulation** : Rollout aléatoire jusqu'à la fin
- 4 **Rétropropagation** : Mise à jour des statistiques

Sélection finale : Nœud le plus robuste (meilleure moyenne ou plus visité)

Choix techniques :

- Bibliothèques Python : socket + asyncio



Exigences clés :

- **F26-F27** : Module Bitboard indépendant
- **F32-F37** : IA (Minimax, MCTS, ML)
- **F38-F40** : Réseau (UDP discovery, TCP game, invitations)
- **Tests** : Couverture $\geq 85\%$ (pytest)
- **Documentation** : Sphinx + i18n (EN/FR)